Plotly.js visibility toggles: why they're slow and how to make them faster

Your **100-300ms lag is inherent to Plotly's architecture**, not a bug or misconfiguration. Plotly.js uses a non-reactive rendering pipeline where even simple visibility changes trigger expensive recalculation cycles—the library wasn't designed for high-FPS interactions. GitHub However, several optimization strategies can dramatically reduce this overhead, including CSS-based fast toggling with state synchronization, which can achieve near-instant updates for your use case.

The core problem: Plotly diffs against its internal state, not the DOM, so every `relayout()` call goes through validation, defaults computation, and potential axis recalculation regardless of what actually changed. GitHub GitHub For your 10 shapes and 11 annotations with paper coordinates on a semi-log chart, the fastest sanctioned approach combines batched `Plotly.relayout()` calls with fixed axis ranges. For sub-50ms toggles, a hybrid CSS approach is necessary.

All visibility toggle methods and their syntax

Trace visibility uses `Plotly.restyle()` with three possible values: `true` (visible), `false` (completely hidden), and `'legendonly'` (hidden but appears in legend). GitHub The syntax for toggling specific traces:

```javascript
// Single trace toggle
Plotly.restyle(graphDiv, {visible: false}, [0]);
Plotly.restyle(graphDiv, {visible: 'legendonly'}, [0]);

// Multiple traces with different states
Plotly.restyle(graphDiv, {visible: [true, false, 'legendonly']}, [0, 1, 2]);

// All traces at once (omit third argument)
Plotly.restyle(graphDiv, {visible: false});
```

Shape visibility uses `Plotly.relayout()` with indexed property paths. Shapes **do support** `'legendonly'` (when `showlegend: true` is set on the shape), allowing them to appear as toggleable legend items:

```javascript
// Single shape
Plotly.relayout(graphDiv, {'shapes[0].visible': false});
Plotly.relayout(graphDiv, {'shapes[2].visible': 'legendonly'});

// Multiple shapes in one call (critical for performance)
Plotly.relayout(graphDiv, {
  'shapes[0].visible': false,
  'shapes[1].visible': true,
  'shapes[2].visible': false
});
```

Annotation visibility uses the same pattern but **does NOT support** `'legendonly'`—only `true` or `false`:

```javascript
Plotly.relayout(graphDiv, {'annotations[0].visible': false});
Plotly.relayout(graphDiv, {
  'annotations[0].visible': false,
  'annotations[1].visible': true
});
```

Combined updates with `Plotly.update()` handle both data and layout in a single call:

```javascript
Plotly.update(graphDiv,
  {visible: false},                    // trace updates
  {'shapes[0].visible': false, 'annotations[0].visible': true}, // layout updates
  [0]                                  // trace indices
);
```

Why Plotly methods have inherent overhead

The **100-300ms overhead exists because Plotly's architecture treats every update as potentially requiring full recalculation.** When you call `Plotly.relayout()`, the following pipeline executes:

1. **Input validation and alias conversion** — normalizes update object
2. **supplyDefaults** — coerces all layout settings into `gd._fullLayout` with computed defaults
3. **editType flag aggregation** — determines recalculation depth based on which properties changed
4. **Axis recalculation** — if autorange is enabled, computes axis extents from all visible data
5. **Component redraw** — shapes, annotations, and images are redrawn
6. **DOM updates** — SVG elements are created/modified/removed

The **editType system** theoretically enables fast paths: properties are tagged with values like `calc` (full recalculation), `plot` (redraw without calc), `ticks` (axis ticks only), or `arraydraw` (individual component update). GitHub However, **visibility changes are tagged with `editType: 'calc'`**—the most expensive tier—because the library must recalculate which traces affect axis autorange, legend entries, and subplot composition. GitHub

From Plotly GitHub Issue #5522: *"These APIs weren't designed for high FPS interactions, each call can trigger a lot of recalculations, whether they're needed or not."* GitHub github The architecture maintains internal state objects (`gd._fullData`, `gd._fullLayout`, `gd.calcdata`) and re-renders from these, never reading the current DOM state.

Fast paths and optimized code paths

Plotly does NOT have an optimized fast path for visibility-only changes. Despite the editType system, visibility toggles trigger full calculation cycles. The only documented fast paths are:

- `axrange` **editType** — optimized for range-only changes during zoom/pan GitHub
- `ticks` **editType** — fast path for tick/gridline updates
- `redraw: false` **in animations** — skips post-transition full redraw (significant performance gain)

For animations, setting `redraw: false` can provide major speedups:

```javascript
Plotly.animate(graphDiv, frameData, {
  transition: { duration: 0 },
  frame: { duration: 50, redraw: false }  // Skips full redraw
});
```

However, this only works for traces during animation frames, not for layout elements like shapes and annotations.

The visible property: behavior differences

Performance of `visible: false` vs removing from array:

Approach	Performance	State preservation	Index stability
`visible: false`	Faster	Element preserved	Indices stable
Array removal	Slower	Element destroyed	Indices shift

Setting `visible: false` is faster and preserves the element for later re-showing. Removing elements causes array re-indexing which breaks hardcoded index references:

```javascript
// Preferred: toggle visibility
Plotly.relayout(gd, {'shapes[1].visible': false});

// Avoid: removal (indices 2+ become 1+)
Plotly.relayout(gd, {'shapes[1]': 'remove'});
```

`'legendonly'` support:

- **Traces**: ✅ Supported — trace hidden but appears in clickable legend
- **Shapes**: ✅ Supported (requires `showlegend: true` on shape)
- **Annotations**: ❌ NOT supported — boolean only

Internal handling differs: Trace visibility affects axis autorange calculation (Plotly must determine min/max of all visible traces). Shape/annotation visibility with paper coordinates (`xref: 'paper'`) doesn't require axis recalculation, but still goes through the full `relayout` pipeline.

Direct SVG manipulation: possibilities and risks

Plotly does NOT expose a documented API for SVG access, but the DOM structure is accessible. The SVG hierarchy for a typical chart:

```
.main-svg
 ├── .layer-below > .shapelayer     (shapes with layer:'below')
 ├── .cartesianlayer > .subplot
 │    └── .layer-subplot > .shapelayer (shapes with layer:'between')
 ├── .infolayer                     (annotations, legend, title)
 └── .layer-above > .shapelayer     (shapes with layer:'above')
```

`Medium`

CSS visibility works immediately:

```javascript
// Fast: ~0ms, but unsupported
document.querySelectorAll('.shapelayer path')[0].style.display = 'none';
```

Critical warning: Plotly diffs against internal state (`gd._fullLayout`), NOT the DOM. Any subsequent `Plotly.relayout()`, `Plotly.react()`, or even zoom/pan interactions will **overwrite your CSS changes** when those elements are redrawn. GitHub The safest pattern hooks into `plotly_afterplot`:

```javascript
let cssVisibility = { shapes: [true, true, false], annotations: [true, false] };

myPlot.on('plotly_afterplot', () => {
 // Re-apply CSS visibility after every Plotly render
 document.querySelectorAll('.shapelayer path').forEach((el, i) => {
  el.style.display = cssVisibility.shapes[i] ? '' : 'none';
 });
});
```

Identifying specific SVG elements: No stable IDs or data attributes link SVG elements to their array indices. You must rely on DOM order matching array order (fragile) or use `nth-child` selectors:

```javascript
// Shape at index 2 (assuming layer:'above')
document.querySelector('.layer-above .shapelayer path:nth-child(3)');
```

Export caveat: `Plotly.downloadImage()` uses internal state, not DOM—CSS-hidden elements will appear in exported images.

Batch updates and performance characteristics

Plotly does NOT automatically batch multiple API calls. Sequential `relayout()` calls each trigger independent recalculation cycles: GitHub

```javascript
// BAD: 3 separate recalculations (~300-900ms total)
Plotly.relayout(gd, {'shapes[0].visible': false});
Plotly.relayout(gd, {'shapes[1].visible': true});
Plotly.relayout(gd, {'annotations[0].visible': false});

// GOOD: 1 recalculation (~100-300ms total)
Plotly.relayout(gd, {
  'shapes[0].visible': false,
  'shapes[1].visible': true,
  'annotations[0].visible': false
});
```

`Plotly.react()` vs `Plotly.relayout()` for visibility:

Method	Use case	Performance
`Plotly.relayout()`	Partial layout updates	Fastest for layout-only changes
`Plotly.react()`	Full figure replacement	Comparable speed, but requires array identity change or `datarevision` bump
`Plotly.update()`	Combined data + layout	Single call when both change
`Plotly.newPlot()`	Full recreation	Slowest (destroys/recreates)

No API exists to skip layout recalculation. There is no `skipValidation` flag or CSS-only mode in the public API.

Configuration options that reduce overhead

`staticPlot: true` does NOT speed up programmatic updates—it only disables user interactions (hover, zoom, pan). The internal recalculation pipeline runs identically.

Effective optimizations:

```javascript
const config = {
  responsive: false,  // Prevents resize recalculations
  staticPlot: false,  // Only if you need interactivity
};

const layout = {
  // CRITICAL: Fix axis ranges to avoid autorange recalculation
  xaxis: { range: [1e-3, 1e3], type: 'log', autorange: false },
  yaxis: { range: [0, 100], autorange: false },

  // Hide legend if frequently updating (legend drawing is expensive)
  showlegend: false,

  // Paper coordinates avoid axis-dependent calculations
  shapes: [{
    xref: 'paper', yref: 'paper',  // You're already using this ✓
    x0: 0.1, y0: 0, x1: 0.1, y1: 1,
    visible: true
  }]
};
```

Your paper coordinates (`xref: 'paper'`) are already optimal—they don't require axis-to-pixel conversion and won't trigger axis recalculation. DeepWiki The semi-log scale adds minimal overhead for shapes since paper coordinates bypass axis transformation entirely.

Recommended high-performance toggle pattern

For your specific case (10 shapes, 11 annotations, paper coordinates), here's the **hybrid approach** that achieves near-instant toggles while maintaining Plotly state integrity:

javascript

```javascript
class FastPlotlyToggle {
 constructor(graphDiv) {
  this.gd = graphDiv;
  this.shapeVisibility = new Array(10).fill(true);
  this.annotationVisibility = new Array(11).fill(true);

  // Re-apply CSS state after any Plotly render
  this.gd.on('plotly_afterplot', () => this.applyCSSState());
 }

 // FAST: CSS toggle (~0-5ms)
 toggleShapeCSS(index, visible) {
  this.shapeVisibility[index] = visible;
  const shapes = this.gd.querySelectorAll('.shapelayer path');
  if (shapes[index]) {
   shapes[index].style.display = visible ? '' : 'none';
  }
 }

 // FAST: CSS toggle for annotations
 toggleAnnotationCSS(index, visible) {
  this.annotationVisibility[index] = visible;
  const annotations = this.gd.querySelectorAll('.annotation');
  if (annotations[index]) {
   annotations[index].style.display = visible ? '' : 'none';
  }
 }

 // Re-apply CSS state after Plotly redraws
 applyCSSState() {
  const shapes = this.gd.querySelectorAll('.shapelayer path');
  shapes.forEach((el, i) => {
   if (i < this.shapeVisibility.length) {
    el.style.display = this.shapeVisibility[i] ? '' : 'none';
   }
  });

  const annotations = this.gd.querySelectorAll('.annotation');
  annotations.forEach((el, i) => {
   if (i < this.annotationVisibility.length) {
    el.style.display = this.annotationVisibility[i] ? '' : 'none';
   }
  });
 }

 // SYNC: Before any Plotly API call, sync state (~100-300ms)
 syncToPlotly() {
```

```
    const updates = {};
    this.shapeVisibility.forEach((vis, i) => {
      updates[`shapes[${i}].visible`] = vis;
    });
    this.annotationVisibility.forEach((vis, i) => {
      updates[`annotations[${i}].visible`] = vis;
    });
    return Plotly.relayout(this.gd, updates);
  }
}

// Usage:
const toggle = new FastPlotlyToggle(myPlot);

// Instant UI response
toggle.toggleShapeCSS(0, false);  // ~0ms
toggle.toggleAnnotationCSS(3, true);  // ~0ms

// Before zoom, export, or other Plotly operations
await toggle.syncToPlotly();  // ~150ms, but only when needed
```

When to sync to Plotly state:

- Before `Plotly.downloadImage()` or `Plotly.toImage()`
- Before programmatic zoom/pan via `Plotly.relayout()`
- Before adding/removing shapes or annotations
- Before any operation that triggers a redraw

This pattern gives you **instant visual feedback** for user toggles while maintaining the ability to properly interact with Plotly's API when needed. The 100-300ms cost is deferred until absolutely necessary rather than paid on every toggle.

Conclusion

The fundamental limitation is architectural: Plotly.js wasn't designed for high-frequency visibility toggles, and there's no internal fast path for these operations. `GitHub` Your options in order of performance:

1. **CSS manipulation with `plotly_afterplot` sync** — ~0-5ms toggles, requires state management
2. **Batched `Plotly.relayout()` with fixed ranges** — ~100-200ms, fully supported
3. **Standard `Plotly.relayout()` per element** — ~100-300ms × number of calls, avoid

For your 10 shapes + 11 annotations, the hybrid CSS approach is the only way to achieve sub-50ms response times. The paper coordinates you're using are already optimal—they don't contribute to the overhead. Ensure `autorange: false` on both axes and batch all visibility changes into single API calls when you do need to use Plotly's methods.